

LANGCHAIN MASTERY

The Complete LangChain Cheatsheet

Every component, every concept,
one real app built step-by-step.

AI Coach John PROITBRIDGE Follow

Repost

CHEATSHEET

What is LangChain?

WHAT IT IS

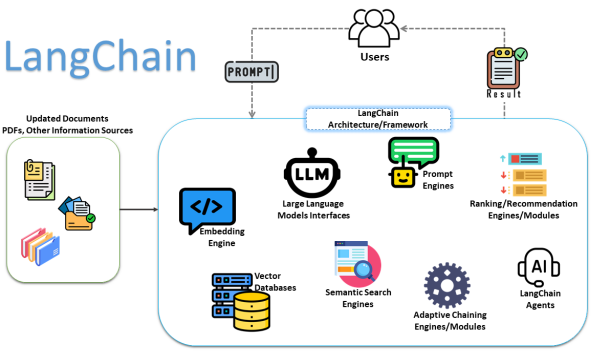
LangChain is an open-source framework for building applications powered by LLMs. It doesn't replace the model — it gives you the plumbing around it: connecting prompts, models, data sources, memory, and tools into one coherent pipeline.

THE CORE IDEA

Instead of writing custom glue code every time you connect an LLM to a database, an API, or a document store, LangChain gives you standardized, reusable building blocks (called Runnables) that snap together.

IN ONE LINE

If the LLM is the engine, LangChain is the car — the chassis, wiring, and controls that turn raw model power into a working application.



AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

Why Do We Need LangChain?

THE PROBLEM

Raw LLM API calls are stateless, have no memory, can't access your private data, and can't take real-world actions on their own.

WHAT LANGCHAIN SOLVES

Connects LLMs to your own documents/data (RAG)
Adds memory across conversation turns
Lets LLMs call external tools and APIs
Standardizes prompt → model → output pipelines
Makes multi-step, multi-model workflows composable

EXAMPLES

A customer support bot that answers from your company docs (RAG)
An agent that checks weather, books a flight, and emails confirmation (tools)
A chatbot that remembers your name across 20 messages (memory)


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

LangChain Architecture Overview

THE BIG PICTURE

Every LangChain app is built from a combination of these core pieces:
Models — the LLM or chat model
Prompts — structured input templates
Output Parsers — structured output extraction
Runnables / LCEL — the composition layer connecting everything
Memory — conversational state
Document Loaders + Splitters — ingesting external data
Embeddings + Vector Stores — semantic search
Retrievers — fetching relevant context
Chains — multi-step pipelines
Agents + Tools — autonomous decision-making
Callbacks — logging, monitoring, streaming
We'll go through each one, then build a real app using them.


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

Models

WHAT

The interface to the actual LLM — LangChain supports two types: LLM (text-in, text-out) and ChatModel (message-in, message-out — what you'll use 95% of the time).

KEY CLASSES

ChatOpenAI, ChatAnthropic, ChatGoogleGenerativeAI, etc.

Code Explained:

We instantiate ChatAnthropic as our model, implementing the standard Runnable interface. Swapping this for ChatOpenAI is a one-line change.

```
from langchain_anthropic import ChatAnthropic
model = ChatAnthropic(model="claude-sonnet-4-6")
response = model.invoke("Explain RAG in one line")
print(response.content)
```

WHY IT MATTERS: Swapping models is a one-line change — your prompts, chains, and logic stay identical regardless of provider.

CHEATSHEET

Prompts

WHAT

Templates that structure input to the model — instead of hardcoding strings, you define reusable templates with variables.

KEY CLASSES

PromptTemplate (single string), ChatPromptTemplate (multi-message, with roles)

Code Explained:

We define a template with placeholders like {role} and {question}. .invoke() fills these in before passing the prompt to the model.

```
from langchain_core.prompts import ChatPromptTemplate
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful {role}."),
    ("human", "{question}")
])
formatted = prompt.invoke({"role": "tutor", "question": "What is a
vector?"})
```

WHY IT MATTERS: Prompts become parameterized, testable, and reusable — not scattered string concatenation across your codebase.

CHEATSHEET

Output Parsers

WHAT

Convert raw LLM text output into structured data your code can actually use — strings, JSON, lists, or custom objects.

KEY CLASSES

StrOutputParser, JsonOutputParser, PydanticOutputParser

Code Explained:

We pipe prompt | model | parser. The parser takes raw text from the LLM and formats it for downstream code.

```
from langchain_core.output_parsers import StrOutputParser
parser = StrOutputParser()
chain = prompt | model | parser
result = chain.invoke({"role": "tutor", "question": "What is a
vector?"})
```

WHY IT MATTERS: Without parsing, you're stuck doing fragile manual string manipulation on raw LLM output. Parsers make outputs reliable for downstream code.


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

Runnables & LCEL (the heart of LangChain)

WHAT

LCEL (LangChain Expression Language) is the | pipe syntax that chains components together. Every component — prompts, models, parsers, retrievers — implements the Runnable interface, meaning they all share the same .invoke(), .stream(), .batch() methods.

Code Explained:

LCEL uses the | operator to chain components. Every part supports .invoke(), .stream(), and .batch() natively.

```
chain = prompt | model | parser
chain.invoke({"role": "tutor", "question": "Explain embeddings"})
chain.stream({"role": "tutor", "question": "Explain embeddings"})  #
streaming
chain.batch([{"role": "tutor", "question": "Q1"}, {"role": "tutor",
"question": "Q2"}])
```

WHY IT MATTERS: This unified interface is what makes LangChain composable — any Runnable can be swapped, combined, or run in parallel using the same syntax.


AI Coach John PROITBRIDGE Follow

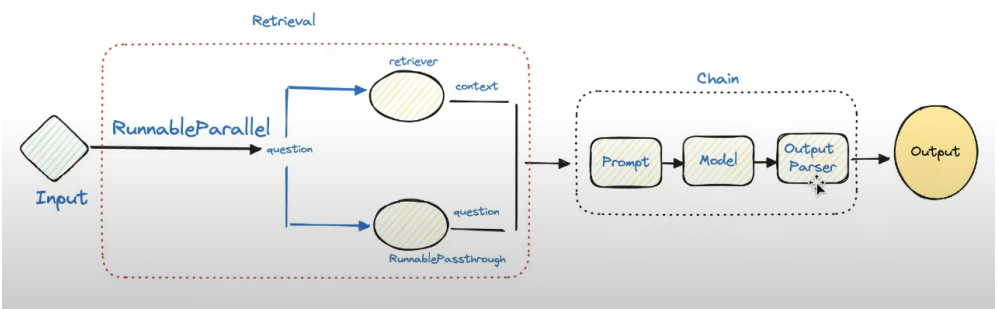
Repost

CHEATSHEET

Runnable Variants

VARIANTS

RunnableParallel — run multiple chains simultaneously, combine results
RunnableLambda — wrap any custom Python function into the pipeline
RunnablePassthrough — pass input through unchanged, often used to preserve original data alongside transformed data in RAG chains.



WHY IT MATTERS: These let you build non-linear pipelines — parallel branches, custom logic injection, and data preservation — all within the same LCEL syntax.


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

Memory

WHAT

Gives your chain awareness of previous conversation turns — without it, every .invoke() call is stateless and isolated.

KEY APPROACHES

Modern LangChain favors RunnableWithMessageHistory wrapping a chain, backed by a message store (in-memory, Redis, etc.)

Code Explained:

We wrap our base chain with RunnableWithMessageHistory. It intercepts every input, fetches past messages, and appends them.

```
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import
RunnableWithMessageHistory
store = {}
def get_history(session_id):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]
chain_with_memory = RunnableWithMessageHistory(chain, get_history)
```

WHY IT MATTERS: Without memory, your chatbot forgets the user's name the moment they say it. Memory makes multi-turn conversation possible.


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

Document Loaders

WHAT

Pull data from external sources (PDFs, websites, databases, Notion, etc.) into LangChain's standard Document format.

KEY CLASSES

PyPDFLoader, WebBaseLoader, CSVLoader, DirectoryLoader

Code Explained:

PyPDFLoader takes a file path and converts it into Langchain Document objects, containing text and metadata.

```
from langchain_community.document_loaders import PyPDFLoader
loader = PyPDFLoader("company_policy.pdf")
docs = loader.load()
```

WHY IT MATTERS: This is step one of any RAG pipeline — without loaders, your LLM has no way to "read" your actual data sources in a standardized format.


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

Text Splitters

WHAT

Break large documents into smaller chunks that fit within context limits and retrieve cleanly — the chunking step we covered extensively in RAG.

KEY CLASSES

RecursiveCharacterTextSplitter (most common, splits intelligently by paragraph → sentence → word)

Code Explained:

RecursiveCharacterTextSplitter slices documents into 500-character chunks with a 50-character overlap to preserve context.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=50)
chunks = splitter.split_documents(docs)
```

WHY IT MATTERS: Bad chunking breaks retrieval quality — this is the single most impactful preprocessing step in any RAG pipeline.


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

Embeddings

WHAT

Converts text chunks into numeric vectors capturing semantic meaning, used for similarity search.

KEY CLASSES

OpenAIEmbeddings, HuggingFaceEmbeddings, VoyageAIEmbeddings

Code Explained:

OpenAIEmbeddings converts text chunks into high-dimensional vectors, which can be mathematically compared for semantic similarity.

```
from langchain_openai import OpenAIEmbeddings
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vector = embeddings.embed_query("What is LangChain?")
```

WHY IT MATTERS: Embeddings are what make semantic search possible — they're the foundation every vector store and retriever is built on.


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

Vector Stores

WHAT

Databases optimized to store embeddings and perform fast similarity search over them.

KEY CLASSES

Chroma, FAISS, Pinecone, Qdrant

Code Explained:

We initialize a Chroma vector store directly from our document chunks and embedding model.

```
from langchain_chroma import Chroma
vectorstore = Chroma.from_documents(chunks, embedding=embeddings)
```

WHY IT MATTERS: This is where your processed, embedded knowledge actually lives — the searchable index your retriever queries against.


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

Retrievers

WHAT

The interface that takes a query and returns relevant documents from a vector store — wraps the vector store with extra retrieval logic.

Code Explained:

vectorstore.as_retriever() creates a Runnable that, when queried, returns the top k most relevant chunks.

```
retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
relevant_docs = retriever.invoke("What is the refund policy?")
```

ADVANCED VARIANTS

MultiQueryRetriever (query expansion), ContextualCompressionRetriever (reranking/compression)

WHY IT MATTERS: Retrievers are also Runnables — meaning they snap directly into LCEL chains alongside prompts and models.


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

Chains (Pre-built Patterns)

WHAT

Common multi-step LLM workflows, either built manually with LCEL or via LangChain's higher-level helper functions.

KEY EXAMPLE

RAG Chain

Code Explained:

create_retrieval_chain combines a document-stuffing chain with our retriever to automate RAG.

```
from langchain.chains.combine_documents import
create_stuff_documents_chain
from langchain.chains import create_retrieval_chain
doc_chain = create_stuff_documents_chain(model, prompt)
rag_chain = create_retrieval_chain(retriever, doc_chain)
rag_chain.invoke({"input": "What is the refund policy?"})
```

WHY IT MATTERS: Chains save you from re-writing common patterns like "retrieve, stuff into prompt, generate" from scratch every time.

CHEATSHEET

Tools

WHAT

Functions an LLM can call to interact with the outside world — APIs, calculators, search engines, databases.

Code Explained:

We define get_weather and use the @tool decorator. LangChain inspects the signature to expose it to the LLM.

```
from langchain_core.tools import tool
@tool
def get_weather(city: str) -> str:
    """Get current weather for a city."""
    return f"It's sunny in {city}"
```

WHY IT MATTERS: Tools are what turn a passive text generator into something that can take real actions — the foundation of agentic behavior.


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

Agents

WHAT

LLM-driven systems that decide which tools to call, in what order, based on reasoning — not a fixed predefined sequence.

Code Explained:

We pass our model and tools into an AgentExecutor. The agent autonomously determines which tools to call.

```
from langchain.agents import create_react_agent, AgentExecutor
agent = create_react_agent(model, tools=[get_weather],
prompt=agent_prompt)
executor = AgentExecutor(agent=agent, tools=[get_weather])
executor.invoke({"input": "What's the weather in Hyderabad?"})
```

WHY IT MATTERS: Agents handle open-ended tasks where the steps aren't known in advance — the model reasons about what to do next at each step.


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

Callbacks

WHAT

Hooks into the execution lifecycle of a chain — used for logging, streaming tokens, monitoring costs, and debugging.

Code Explained:

StdOutCallbackHandler hooks into the chain's execution to print intermediate steps, API calls, and raw outputs.

```
from langchain_core.callbacks import StdOutCallbackHandler
chain.invoke({"question": "Explain RAG"}, config={"callbacks":
[StdOutCallbackHandler()]})
```

WHY IT MATTERS: In production, callbacks are how you get observability. By using the StdOutCallbackHandler, we can see the exact input and output at every stage of the chain. This is crucial for debugging complex agent workflows where the LLM might make multiple hidden API calls before returning a final answer to the user.


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

LangGraph (Brief Mention)

WHAT

LangChain's framework for building more complex, stateful, cyclical agent workflows — when simple linear chains or basic agents aren't enough (e.g., multi-agent systems, loops, human-in-the-loop approval steps).

WHEN TO USE

Simple Q&A or RAG → stick with LCEL chains.
Complex multi-step agent workflows with branching/looping logic → LangGraph.

WHY IT MATTERS: While standard LangChain LCEL is perfect for predictable, linear workflows, LangGraph introduces cycles and state. This allows you to build autonomous agents that can loop, self-correct, and pause for human approval before executing dangerous tools.


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

BUILD: Project Overview

WHAT WE'RE BUILDING

A simple RAG-powered Q&A app over a company FAQ document — using nearly every component covered so far.

FLOW

Load document → Split into chunks → Embed → Store in vector DB → Retrieve relevant chunks → Generate grounded answer → Add memory for follow-up questions

STACK

LangChain + Anthropic Claude + Chroma vector store
Let's build it step by step ➡️


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

Step 1: Load & Split the Document

```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
loader = TextLoader("faq.txt")
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=50)
chunks = splitter.split_documents(docs)
print(f"Loaded {len(chunks)} chunks")
```

WHAT'S HAPPENING: We load the raw FAQ file and break it into retrieval-friendly chunks. The 50-character overlap is critical: if a concept spans across the end of chunk A and the beginning of chunk B, the overlap ensures the LLM still gets the full context regardless of which chunk is retrieved.

CHEATSHEET

Step 2: Embed & Store in Vector DB

```
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = Chroma.from_documents(chunks, embedding=embeddings)
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
```

WHAT'S HAPPENING: Each text chunk is converted into a high-dimensional vector using OpenAI's embedding model, and saved into a Chroma database. The retriever is configured with `k=3`, meaning it will only fetch the top 3 most semantically similar chunks when a user asks a question, saving tokens and improving accuracy.


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

Step 3: Build the Prompt & Model

```
from langchain_anthropic import ChatAnthropic
from langchain_core.prompts import ChatPromptTemplate
model = ChatAnthropic(model="claude-sonnet-4-6")
prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer only using the provided context.
Context:\n{context}"),
    ("human", "{input}")
])
```

WHAT'S HAPPENING: We define our Claude 3.5 Sonnet model and a strict system prompt. Grounding the prompt with the `{context}` variable forces the LLM to strictly use the retrieved chunks to answer the user's question. This is the core mechanism that prevents the model from hallucinating.

CHEATSHEET

Step 4: Assemble the RAG Chain

```
from langchain.chains.combine_documents import
create_stuff_documents_chain
from langchain.chains import create_retrieval_chain
doc_chain = create_stuff_documents_chain(model, prompt)
rag_chain = create_retrieval_chain(retriever, doc_chain)
response = rag_chain.invoke({"input": "What is your refund
policy?"})
print(response["answer"])
```

WHAT'S HAPPENING: The `create_stuff_documents_chain` stuffs the retrieved documents into the prompt, while the `create_retrieval_chain` handles the actual search against the vector database. This wires everything together so a single `.invoke()` triggers the entire pipeline seamlessly.

CHEATSHEET

Step 5: Add Memory for Follow-Up Questions

```
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import
RunnableWithMessageHistory
store = {}
def get_history(session_id):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]
conversational_rag = RunnableWithMessageHistory(
    rag_chain, get_history,
    input_messages_key="input",
    history_messages_key="chat_history"
)
```

WHAT'S HAPPENING: We wrap our RAG chain in `RunnableWithMessageHistory`. This intercepts every incoming question, fetches the past chat history from a session-specific store, and appends it to the prompt. Now the LLM can resolve pronouns like "it" or "that" based on prior conversation turns.


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

Step 6: Run the Full App

```
config = {"configurable": {"session_id": "user_123"}}
r1 = conversational_rag.invoke({"input": "What is your refund
policy?"}, config=config)
print(r1["answer"])
r2 = conversational_rag.invoke({"input": "Does that apply to
international orders too?"}, config=config)
print(r2["answer"])
```

WHAT'S HAPPENING: Because we attached memory, the second question doesn't need to explicitly mention "refund policy". The chain understands it's a follow-up. This complete, production-ready pattern uses loaders, splitters, embeddings, vector stores, and memory all working in perfect harmony.


AI Coach John PROITBRIDGE Follow Repost

CHEATSHEET

Quick Reference: Component Cheatsheet

Model	Talk to the LLM
Prompt	Structure input
Output Parser	Structure output
Runnable/LCEL	Compose everything
Memory	Persist conversation state
Document Loader	Ingest external data
Text Splitter	Chunk documents
Embeddings	Convert text → vectors
Vector Store	Store & search vectors
Retriever	Fetch relevant chunks
Chain	Multi-step pipeline
Tool	Action the LLM can call
Agent	Reasoning + tool use
Callback	Logging/monitoring
LangGraph	Complex stateful workflows


AI Coach John PROITBRIDGE Follow Repost